SIMATS ENGINEERING
ASSESSMENT TOOL

COURSE NAME: Theory of Computation **COURSE CODE: CSA1308**

COURSE OUTCOME COVERED

CO1: *Analyze fundamental mathematical concepts of automata theory for formal languages and decision problems(BL:3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:25**
Duration : 1 Hour

Simulation

Code of Conduct:

I ***D.Naveen (192372177)*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SOLUTIONS

Course: Theory of Computation (CSA1308) | CO1 | Student: **D.Naveen (192372177)**

Notation used: Q = set of states, Σ = input alphabet, δ = transition function, q_0 = start state, F = set of final (accepting) states. In tables, "→" marks the start state and "*" marks a final state. In diagrams, a double circle is a final state and the unlabeled arrow marks the start state.

Question 1 — NFA for an Automatic Door System (5 Marks)

Question: *Design a Non-Deterministic Finite Automaton (NFA) for an automatic door with four states — Closed, Opening, Open, Closing — and input symbols P (person detected) and N (no person detected). The door starts Closed; on P it may non-deterministically go from Closed to Opening; once opened it moves to Open; on N it goes to Closing and eventually returns to Closed. Non-determinism: the door may stay Open even when a person is detected, or start closing when no one is detected.*

1. Formal definition

The machine is the 5-tuple M = (Q, Σ, δ, q_0, F) where:

• Q = {Closed, Opening, Open, Closing} • Σ = {P, N} • q_0 = Closed • F = {Closed} (a complete door cycle ends with the door shut again)

• δ : Q × Σ → 2^Q (each entry is a *set* of possible next states):

δ	P (person detected)	N (no person)
→ * Closed	{Closed, Opening}	{Closed}
Opening	{Open}	{Open}
Open	{Open}	{Open, Closing}
Closing	{Opening}	{Closed}

Where the non-determinism is: δ(Closed, P) has two choices (stay Closed, or begin Opening) and δ(Open, N) has two choices (stay Open, or begin Closing). These are the two dashed red arrows in the diagram. The extra arrow Closing → Opening on P is a safety rule: a person detected while the door is closing makes it re-open.

2. Transition diagram

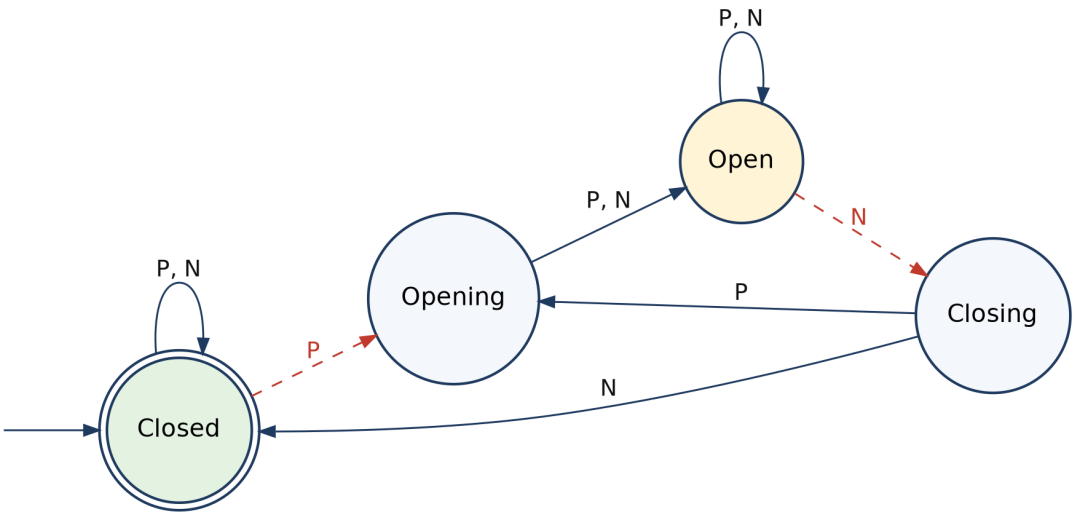


Fig. 1 — NFA for the automatic door (dashed red = non-deterministic choices)

3. Simulation of the input P P N N

Intended behaviour (one computation path): Closed —P→ Opening —P→ Open —N→ Closing —N→ Closed, so the door completes a full cycle and the string is accepted. Because the NFA explores all choices at once, the set of possible states after each symbol is:

After reading	Set of possible states
(start)	{Closed}
P	{Closed, Opening}
P P	{Closed, Opening, Open}
P P N	{Closed, Open, Closing}
P P N N	{Closed, Open, Closing}

Final set contains **Closed** ∈ F, so **PPNN is accepted**. A string such as "P P" can also be followed along the path Closed → Opening → Open (door left open), which shows the machine does not need to decide in advance which choice to take — an NFA accepts if *at least one* path reaches a final state.

Conclusion: The NFA models all four door states and both sources of non-determinism, and its Closed → Opening → Open → Closing → Closed cycle matches the specified behaviour.

Question 2 — Finite Automaton for a Traffic Light Controller (5 Marks)

Question: *Design a finite automaton for a traffic light at an intersection with states Red, Green and Yellow, transitions driven by a timer/control signal in the order Red → Green → Yellow → Red. Include timing constraints, explain continuous cycling, and explain how the model extends to pedestrian crossing signals and emergency vehicle override.*

1. Formal definition (Moore machine – output depends on state)

• Q = {R (Red), G (Green), Y (Yellow)} • Σ = {t, w}, where **t** = timer expired, **w** = timer still running (wait)
• q_0 = R • F = ∅ — the controller never halts, so no accepting state is needed.

Output (lamp shown) = Red in R, Green in G, Yellow in Y. Timing constraint: the timer is loaded with the state's duration on entry and raises **t** when it reaches zero. Typical values are given below (they can be changed for a real junction).

State	Lamp lit	Duration	On t	On w
R (start)	Red	30 s	→ G	stay in R
G	Green	25 s	→ Y	stay in G
Y	Yellow	5 s	→ R	stay in Y

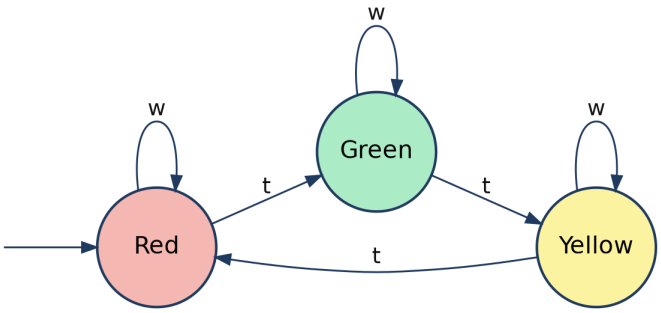


Fig. 2 — Basic traffic-light automaton (w = wait loops, t = timer expiry)

2. Continuous cycling and safety

The transition graph is a single cycle R → G → Y → R, so after Y the timer is reloaded and the automaton returns to R automatically; the cycle length is 30 + 25 + 5 = 60 s and repeats indefinitely. Safety follows from the structure: there is no arrow G → R or R → Y, so Green can never jump directly to Red and Yellow always warns before Red. The self-loops on w keep the current lamp on until the timer expires. For a real two-road crossing, two copies run in anti-phase (one road Green while the other is Red).

3. Extension: pedestrian crossing and emergency override

Add new input signals **p** = pedestrian button pressed (latched), **e** = emergency vehicle detected, **c** = emergency cleared, and two new states **W** (Pedestrian Walk: all vehicle lamps Red, walk signal ON) and **E** (Emergency: lamps set to clear the emergency vehicle's route / all others Red).

State	t, no p	t, p	e	c
→ R	G	W	E	—
G	Y	Y	E	—
Y	R	R	E	—
W (ped. walk)	G	G	E	—
E (emergency)	—	—	E	R

"—" means the input is ignored (self-loop). In practice the controller passes through a short Yellow/all-red clearance interval before entering E so that traffic already in the junction can clear.

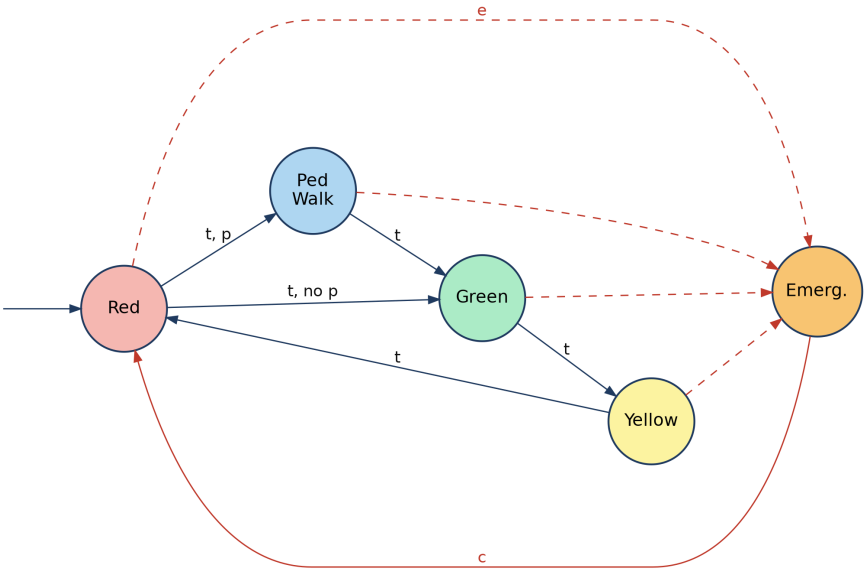


Fig. 3 — Extended automaton (red dashed arrows: e from every state to E; c returns to R)

Why this is safe and efficient: pedestrians only get Walk in state W, when every vehicle lamp is Red; emergency override is reachable from every state but is always left through R, so normal ordering resumes cleanly; and because the pedestrian phase is entered only when p is latched, no green time is wasted when nobody is waiting.

Question 3 — DFA accepting strings that end with "ab" over {a, b} (5 Marks)

Question: *Design a DFA using a simulator to accept the strings that end with **ab** over Σ = {a, b}. Test input: W = aaabab.*

1. Idea

The DFA only needs to remember how much of the ending "ab" has been seen most recently: **q0** = nothing useful yet (last symbol b, or no input), **q1** = the last symbol was a, **q2** = the last two symbols were "ab" (accepting).

2. Formal definition

Q = {q0, q1, q2} Σ = {a, b} start state = q0 F = {q2}

δ	a	b
→ q0	q1	q0
q1	q1	q2
* q2	q1	q0

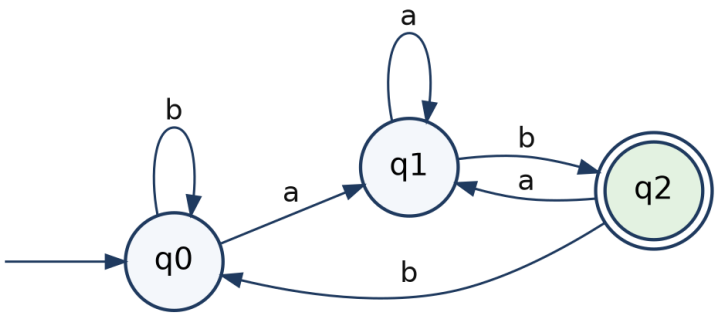


Fig. 4 — DFA for strings ending with ab

3. Simulation of W = aaabab

Step	Current state	Input read	Next state
1	q0	a	q1
2	q1	a	q1
3	q1	a	q1
4	q1	b	q2
5	q2	a	q1
6	q1	b	q2
Result	Input exhausted in **q2** — **ACCEPTED** (final state)		

The DFA finishes in q2, so **aaabab is accepted** — it does end with “ab”.

4. Verification with other inputs

Input string	Path of states	Result
aaabab	q0 → q1 → q1 → q1 → q2 → q1 → q2	Accept
ab	q0 → q1 → q2	Accept
aab	q0 → q1 → q1 → q2	Accept
bab	q0 → q0 → q1 → q2	Accept
a	q0 → q1	Reject
b	q0 → q0	Reject
ba	q0 → q0 → q1	Reject
abb	q0 → q1 → q2 → q0	Reject
aba	q0 → q1 → q2 → q1	Reject
ε	q0	Reject

Steps in the simulator (AutoSim): choose a new Deterministic Finite Automaton; add three states q0, q1, q2; set q0 as the initial state and q2 as final; draw the six transitions from the table (every state has one arrow for a and one for b, so the DFA is complete); then use the simulate/run-input option to enter aaabab and the strings above and compare with the table. The simulator shows the same path q0→q1→q1→q1→q2→q1→q2 and accepts.

Question 4 — DFA accepting only “c”, “bc” and “bcaaa” over {a, b, c} (5 Marks)

Question: *Design a DFA over Σ = {a, b, c} that accepts exactly the strings c, bc and bcaaa; every other string must lead to a dead state. Define states, transitions, start state and accepting states, and verify in a simulator such as AutoSim.*

1. Idea

Each of the three strings follows its own path from the start state, and every state remembers exactly the prefix read so far: q1 = “b”, q2 = “c” (accepting), q3 = “bc” (accepting), q4 = “bca”, q5 = “bcaa”, q6 = “bcaaa” (accepting). Any symbol that does not continue one of these three strings goes to the dead state d, which loops on every symbol.

2. Formal definition

Q = {q0, q1, q2, q3, q4, q5, q6, d} Σ = {a, b, c} start state = q0 F = {q2, q3, q6} (d = dead/trap state)

δ	a	b	c
→ q0	d	q1	q2
q1	d	d	q3
* q2	d	d	d
* q3	q4	d	d
q4	q5	d	d
q5	q6	d	d
* q6	d	d	d
d	d	d	d

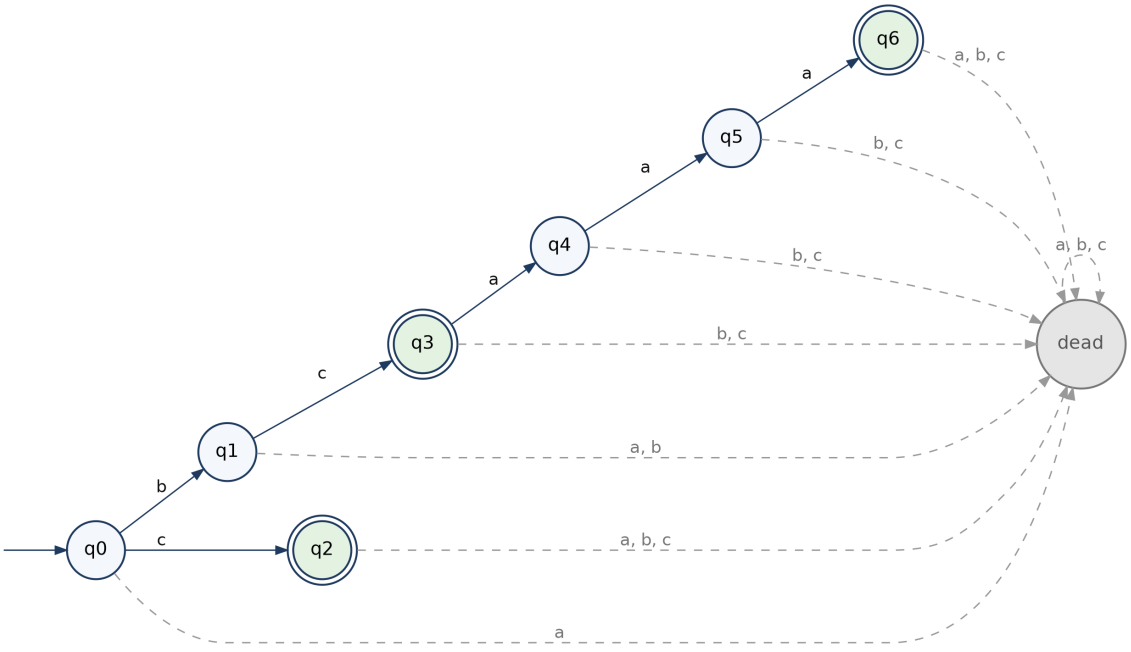


Fig. 5 — DFA for {c, bc, bcaaa} (grey dashed arrows lead to the dead state d)

3. Simulation of the three valid strings

“c” (accepted directly from the start state):

Step	Current state	Input read	Next state
1	q0	c	q2
Result	Input exhausted in **q2** — **ACCEPTED** (final state)		

“bc” (passes through the state for “b”, then “c”):

Step	Current state	Input read	Next state
1	q0	b	q1
2	q1	c	q3
Result	Input exhausted in **q3** — **ACCEPTED** (final state)		

“bcaaa” (tracks each additional a after “bc”):

Step	Current state	Input read	Next state
1	q0	b	q1
2	q1	c	q3
3	q3	a	q4
4	q4	a	q5
5	q5	a	q6
Result	Input exhausted in **q6** — **ACCEPTED** (final state)		

4. Verification with valid and invalid inputs

Input string	Path of states	Result
c	q0 → q2	Accept
bc	q0 → q1 → q3	Accept
bcaaa	q0 → q1 → q3 → q4 → q5 → q6	Accept
b	q0 → q1	Reject
bca	q0 → q1 → q3 → q4	Reject
bcaa	q0 → q1 → q3 → q4 → q5	Reject
bcaaaa	q0 → q1 → q3 → q4 → q5 → q6 → d	Reject
cc	q0 → q2 → d	Reject
cb	q0 → q2 → d	Reject
bb	q0 → q1 → d	Reject
a	q0 → d	Reject
ε	q0	Reject

Steps in the simulator (AutoSim): create a DFA with the 8 states above, mark q0 as initial and q2, q3, q6 as final, and add the transitions from the table (including all arrows into d). Run the strings listed in the verification table: only c, bc and bcaaa end in a final state; all other inputs end in a non-final state or in d and are rejected.

Question 5 — DFA accepting all strings over {a, b} that begin with a or b (5 Marks)

Question: *Design a DFA over Σ = {a, b} that accepts every string that begins with "a" or "b". It should move to an accepting state after the first symbol and stay there whatever follows, with a dead state for invalid or empty input if the simulator requires it.*

1. Idea

Since the alphabet is only {a, b}, every non-empty string starts with a or b, so the language is simply all non-empty strings. q0 is the start state and is **not** accepting, so the empty string is rejected. Reading any first symbol moves to q1, which is accepting and loops on a and b forever.

2. Formal definition

Q = {q0, q1} Σ = {a, b} start state = q0 F = {q1}

δ	a	b
→ q0	q1	q1
* q1	q1	q1

Dead state: not needed here — every state already has a transition for both a and b, so the DFA is complete, and the empty input is rejected because q0 is non-final. (If a simulator insists on a dead state, add a state d with loops on a and b; it would be unreachable and would not change the language.)

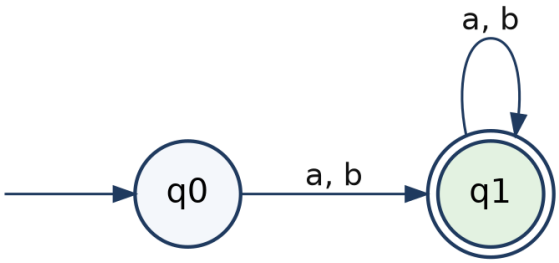


Fig. 6 — DFA accepting every string that begins with a or b

3. Simulation of sample inputs

"abba":

Step	Current state	Input read	Next state
1	q0	a	q1
2	q1	b	q1
3	q1	b	q1
4	q1	a	q1
Result	Input exhausted in **q1** — **ACCEPTED** (final state)		

"b":

Step	Current state	Input read	Next state
1	q0	b	q1
Result	Input exhausted in **q1** — **ACCEPTED** (final state)		

4. Verification

Input string	Path of states	Result
a	q0 → q1	Accept
b	q0 → q1	Accept
ab	q0 → q1 → q1	Accept

Input string	Path of states	Result
ba	q0 → q1 → q1	Accept
aaa	q0 → q1 → q1 → q1	Accept
bbab	q0 → q1 → q1 → q1 → q1	Accept
abba	q0 → q1 → q1 → q1 → q1	Accept
ε	q0	Reject

Steps in the simulator (AutoSim): create a DFA with states q0 (initial) and q1 (final); add the transitions q0 –a,b→ q1 and q1 –a,b→ q1; run the strings above. Every non-empty string is accepted and the empty string is rejected, as the table shows.

All DFAs above were also checked by program against the exact language definition for every string up to length 8–12 over their alphabets, with no mismatches.

SIMATS ENGINEERING
ASSESSMENT TOOL

COURSE NAME: Theory of Computation **COURSE CODE: CSA13**

COURSE OUTCOME COVERED

CO1: *Analyze fundamental mathematical concepts of automata theory for formal languages and decision problems(BL:3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:25**
Duration : 1 Hour

Simulation

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SIMATS ENGINEERING
ASSESSMENT TOOL

1 Design a **Non-Deterministic Finite Automaton (NBA)** for an **Automatic Door System** based on sensor inputs.An automatic door system can be modeled using an NBA to represent different states and transitions. The system consists of four states: *Closed, Opening, Open,* and *Closing*. The input symbols are P (Person detected) and N (No person detected). Initially, the door is in the *Closed* state. When a person is detected (P), the system may non-deterministically transition from *Closed* to *Opening*. Once opened, the system moves to the *Open* state. If no person is detected (N), the door transitions to *Closing* and eventually returns to the *Closed* state. Non-determinism occurs when the system may remain in the *Open* state even if a person is detected or may start closing if no detection occurs.

2 Design a finite automaton to model a traffic light control system at a road intersection. The automaton should represent different states corresponding to the traffic lights, such as Red, Green, and Yellow. Define the transitions between these states based on a timer or control signals, ensuring that the sequence follows the standard order (Red → Green → Yellow → Red). Include considerations for timing constraints, such as how long each light remains active, and describe how the automaton handles continuous cycling of signals. Additionally, explain how this model can be extended to handle pedestrian crossing signals or emergency vehicle overrides, ensuring safe and efficient traffic flow.

3 Design DFA using simulator to accept the string the end with ab over set {a,b)

W= aaabab

4 Design a Deterministic Finite Automaton (DFA) using a simulator to accept only the input strings “c”, “bc”, and “bcaaa” over the alphabet {a, b, c}. The automaton should begin from an initial state and transition through intermediate states based on the sequence of input symbols, ensuring that each valid string follows a unique path. For example, the string “c” should be accepted directly from the start state, while “bc” should pass through states representing “b” and then “c”. Similarly, “bcaaa” should be processed through a sequence of states that track each additional “a” after “bc”. Any deviation from these exact patterns should lead to a dead state, ensuring that no other strings are accepted. Clearly define the states, transition functions, start state, and accepting states, and implement the DFA in a simulator such as Autosim to verify its correctness.

5 Design a Deterministic Finite Automaton (DFA) using a simulator to accept all strings over the alphabet {a, b} that begin with either “a” or “b”. The automaton should start in an initial state and immediately transition to an accepting state upon reading the first input symbol, since any valid string must start with either “a” or “b”. After the first symbol is processed, the DFA should remain in an accepting state regardless of the subsequent sequence of “a” and “b”, thereby allowing any combination of characters to follow. Additionally, include a dead state to handle invalid or empty inputs if required by the simulator. Clearly define the states, transition rules, start state, and accepting states, and verify the design using a simulator such as Autosim to ensure that all valid strings are accepted and invalid ones are rejected.

SIMATS ENGINEERING
ASSESSMENT TOOL

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand